

```
In [ ]: # Initialize Otter
import otter
grader = otter.Notebook("practice04.ipynb")
```

Student names and e-mails:

YOUR NAME — your@calvin.edu

YOUR NAME — your@calvin.edu

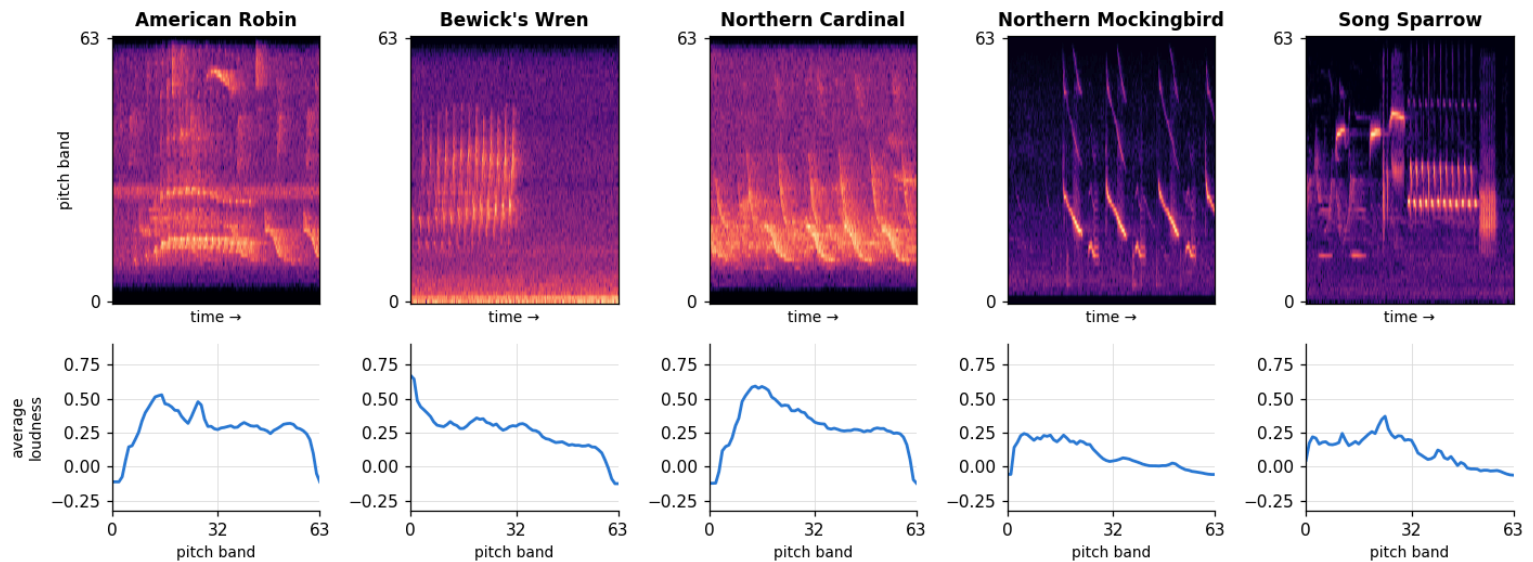
Practice 04 — Clustering and PCA on Birdsong

Five kinds of North American birds, 4,268 short clips of their songs. Can an algorithm that never hears the species names find the species — and if it can't, what *does* it find?

SLO	Description
05A	Apply k-means clustering to group data and interpret the resulting cluster assignments
05B	Evaluate clustering quality using the elbow method and silhouette score to choose an appropriate k
05C	Apply PCA to reduce dimensionality and interpret how much variance each component explains

The Dataset: Birdsong

Top: one clip's spectrogram (pitch x time). Bottom: the same clip averaged over time — the 64 numbers in the dataset.



The recordings come from [Xeno-canto](https://xeno-canto.org/) (<https://xeno-canto.org/>), a community archive of bird sounds, by way of the [BirdCLEF 2021](https://www.kaggle.com/competitions/birdclef-2021) (<https://www.kaggle.com/competitions/birdclef-2021>) machine-learning competition. Five species: American Robin, Bewick's Wren, Northern Cardinal, Northern Mockingbird and Song Sparrow.

Each of 377 recordings was cut into short **clips** — about 11 per recording, 4,268 in all. Each clip was turned into a **spectrogram**: a picture of the sound, with pitch going up and time going across (top row). Then each spectrogram was averaged over time, so every clip becomes **64 numbers** — how loud each of 64 pitch bands is, on average (bottom row). That's the same size as a handwritten digit from class: 64 numbers per row.

Column	What it holds
recording_id	the recording the clip was cut from
band_00 ... band_63	average loudness in each pitch band, lowest pitch first
loudest_band	the band (0–63) with the highest average loudness
common_name , scientific_name	the species — our "answer key," which the algorithms never see
country , recordist	where the recording was made, and by whom
source_url	listen to the whole recording on Xeno-canto
license	the recording's Creative Commons license

Each recording is shared by its recordist under the Creative Commons license in its row; this derived dataset is shared under CC BY-NC-SA 4.0.

Setup (given — just run it)

```
In [ ]: import numpy as np
import pandas as pd
import plotly.express as px
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score

birds = pd.read_csv("https://cs.calvin.edu/courses/data/202/26fa/datasets/birdsong.csv")
bands = [f"band_{i:02d}" for i in range(64)] # the 64 pitch-band columns
X = birds[bands] # what the algorithms see: 64 numbers per clip
birds.head()
```

A First Look (given — just run it)

What does an *average* clip of each species look like? This averages the 64 bands within each species, reshapes the result into long format (Week 4's `melt`), and draws one line per species.

```
In [ ]: profile = birds.groupby("common_name")[bands].mean().reset_index()
profile_long = profile.melt(id_vars="common_name", var_name="band", value_name="loudness")
profile_long["band"] = profile_long["band"].str.replace("band_", "").astype(int)

px.line(profile_long, x="band", y="loudness", color="common_name",
        labels={"band": "pitch band (low → high)", "loudness": "average loudness", "common_name": "species"},
        title="Average pitch profile of each species")
```

The species' profiles differ — but each line is an *average* of hundreds of clips. Individual clips vary a lot more. Keep that in mind as you cluster.

Part 1 — Clustering (SLO 05A)

Task 05A.1 — k-means (2 pts)

Cluster the clips into **5** groups — one per species, if the algorithm could find them — using only the 64 band columns in `X` .

Use `KMeans(n_clusters=5, n_init=10, random_state=42)` , store the fitted model in `kmeans` , and add each clip's cluster number to `birds` as a new column `"cluster"` .

```
In [ ]: kmeans = KMeans(n_clusters=5, n_init=10, random_state=42).fit(X) # SOLUTION
birds["cluster"] = kmeans.labels_ # SOLUTION
birds["cluster"].value_counts()
```

```
In [ ]: grader.check("05A.1")
```

Task 05A.2 — Do the Clusters Match the Species? (2 pts)

Count how many clips of each species (`common_name`) landed in each cluster, with `pd.crosstab(rows, columns)` — rows = clusters, columns = species. Store the table in `species_by_cluster` .

```
In [ ]: species_by_cluster = pd.crosstab(birds["cluster"], birds["common_name"]) # SOLUTION
species_by_cluster
```

```
In [ ]: grader.check("05A.2")
```

Task 05A.3 — What Sets the Clusters Apart? (2 pts)

Two steps:

1. Add a column `mean_loudness` to `birds` : each clip's average over its 64 band columns. That's a mean **across each row**, not down each column — which `axis` is that?
2. Build `cluster_profile` : one row per cluster, with named aggregation (as in Week 3) — `n_clips` (the "count" of `recording_id` : one per clip), `loudest_band` (mean of `loudest_band`) and `mean_loudness` (mean of `mean_loudness`). Sort it by `loudest_band` , lowest first.

```
In [ ]: birds["mean_loudness"] = birds[bands].mean(axis=1) # SOLUTION
cluster_profile = ( # SOLUTION
    # BEGIN SOLUTION NO PROMPT
    birds.groupby("cluster")
    .agg(n_clips=("recording_id", "count"),
         loudest_band=("loudest_band", "mean"),
         mean_loudness=("mean_loudness", "mean"))
    .sort_values("loudest_band")
)
# END SOLUTION
cluster_profile.round(2)
```

```
In [ ]: grader.check("05A.3")
```

Listen! (given — just run it)

The two recordings with the most clips in the cluster with the **lowest** average `loudest_band` , and the two with the most clips in the **highest**. Click each link and listen for a minute. Is the difference between the two clusters the bird that's singing — or something else about the recording?

```
In [ ]: from IPython.display import HTML

low, high = cluster_profile.index[0], cluster_profile.index[-1]
print("lowest-pitch cluster:", low, "   highest-pitch cluster:", high)

examples = (birds[birds["cluster"].isin([low, high])]
            .groupby(["cluster", "common_name", "source_url"], as_index=False)
            .size()                                     # how many clips each recording has
            .sort_values("size", ascending=False)
            .groupby("cluster").head(2))                # the top two recordings of each cluster
HTML(examples.to_html(index=False, render_links=True)) # render_links makes the URLs clickable
```

Part 2 — How Many Clusters? (SLO 05B)

We picked $k = 5$ because we know there are five species. With no labels, would the data itself suggest 5?

Task 05B.1 — Elbow and Silhouette (3 pts)

For $k = 2$ to 10 , fit k-means (same `n_init=10`, `random_state=42`) and store, in order of k , each model's `inertia_` in the list `inertias` and its `silhouette_score(X, km.labels_)` in the list `sil_scores`. Then store the k with the **highest** silhouette score in `best_k`. The plots at the end are given.

Hint: `np.argmax(sil_scores)` gives the *position* of the highest score in the list — and position 0 holds $k = 2$.

```
In [ ]: ks = list(range(2, 11))
inertias, sil_scores = [], []
for k in ks:
    # fit k-means with k clusters (n_init=10, random_state=42)
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X) # SOLUTION
    # add its inertia_ to inertias, and its silhouette score to sil_scores
    inertias.append(km.inertia_) # SOLUTION
    sil_scores.append(silhouette_score(X, km.labels_)) # SOLUTION

best_k = int(np.argmax(sil_scores)) + 2 # SOLUTION
print("best k by silhouette:", best_k)

px.line(x=ks, y=inertias, markers=True,
        labels={"x": "number of clusters k", "y": "inertia"}, title="Elbow method").show()
px.line(x=ks, y=sil_scores, markers=True,
        labels={"x": "number of clusters k", "y": "silhouette score"}, title="Silhouette score").show()
```

```
In [ ]: grader.check("05B.1")
```

Question 05B.2 (1 pt). Does the elbow bend sharply at any k ? Which k has the best silhouette, and how high is that score? What do the two plots together say about how clearly the clips form groups — and about $k = 5$?

Type your answer here, replacing this text.

Sample answer: There is no sharp elbow: the inertia falls smoothly and only flattens gradually around $k = 4$ to 6. The silhouette is highest at $k = 2$, but even that is only about 0.29, and it drops as k grows. Neither plot finds five clear groups: the clips don't fall into well-separated clouds, so any k is a judgment call — five species does not mean five clusters in these 64 numbers.

Part 3 — PCA: 64 Numbers → 2 (SLO 05C)

Task 05C.1 — Two Components (2 pts)

Reduce `X` to 2 components with `PCA(n_components=2, random_state=42)`. Store the 2-D result in `X_2d` and the share of the variance the two components keep (a number between 0 and 1) in `share_2`.

```
In [ ]: pca = PCA(n_components=2, random_state=42) # SOLUTION
X_2d = pca.fit_transform(X) # SOLUTION
share_2 = pca.explained_variance_ratio_.sum() # SOLUTION
print(X_2d.shape, round(share_2, 3))
```

```
In [ ]: grader.check("05C.1")
```

Two Views of the Same Shadow (given — just run it)

The same 2-D picture, colored two ways: by the true species, and by your k-means clusters. Compare them.

```
In [ ]: view = pd.DataFrame({"PC1": X_2d[:, 0], "PC2": X_2d[:, 1],
                             "species": birds["common_name"], "cluster": birds["cluster"].astype
                             (str)})
px.scatter(view, x="PC1", y="PC2", color="species", opacity=0.5,
           title="PCA view, colored by species").show()
px.scatter(view, x="PC1", y="PC2", color="cluster", opacity=0.5,
           title="PCA view, colored by k-means cluster").show()
```

Task 05C.2 — How Many Components? (2 pts)

Fit PCA with **all** components and find the smallest number of components that keeps at least **90%** of the variance. Store it in `n_90`.

Hint: `np.cumsum()` of `explained_variance_ratio_` gives the share kept by the first 1, 2, 3, ... components. Print it and count — or let `np.argmax(cumulative >= 0.9)` find the *position* of the first share that reaches 0.9 (position 0 = 1 component).

```
In [ ]: pca_all = PCA(random_state=42).fit(X) # SOLUTION
        cumulative = np.cumsum(pca_all.explained_variance_ratio_) # SOLUTION
        print(cumulative.round(3))
        n_90 = int(np.argmax(cumulative >= 0.9)) + 1 # SOLUTION
        n_90
```

```
In [ ]: grader.check("05C.2")
```

Question 05C.3 (1 pt). All 64 bands measure loudness on the same scale, but some vary much more than others (standard deviations from about 0.04 to 0.28). Would you use `StandardScaler` before PCA here? What would scaling change?

Type your answer here, replacing this text.

Sample answer: Not required — the bands already share one unit, like the digits' pixels. Scaling would make the quiet, barely-varying bands count as much as the lively ones. Here it changes little (2 components keep about 74% instead of 76%; 6 components for 90% instead of 5). Scale first when columns come in different units.

Submission

Save your notebook, then upload the **.ipynb file** directly to our Moodle assignment page. Don't export, zip, or submit anything else — just the notebook.